

SANOS Y SALVOS

ANALISIS DE PATRONES DE DISENO

Plataforma de reporte y localizacion de mascotas perdidas

DSY1106 - Desarrollo Fullstack III

Evaluacion Parcial N2

2024

1. Introduccion

El proyecto Sanos y Salvos implementa una arquitectura de microservicios compuesta por un BFF (Backend For Frontend) y cuatro microservicios especializados. Esta arquitectura exige aplicar patrones de diseno que promuevan el desacoplamiento, la extensibilidad y la mantenibilidad del sistema.

En este documento se analizan seis patrones GoF implementados en el codebase, indicando el componente que los aplica, el problema que resuelven y las ventajas obtenidas. Ademas se documenta el arquetipo Maven personalizado para la generacion estandarizada de nuevos microservicios.

2. Resumen de patrones implementados

Patron	Categoria	Componente	Archivo principal
Factory Method	Creacional	ms-mascotas	ReporteFactory.java
Observer	Comportamiento	ms-mascotas / ms-geo / ms-coincidencias	EventPublisher / Listeners
Proxy	Estructural	bff-service	AuthProxyService.java
Strategy	Comportamiento	ms-coincidencias / ms-geolocalizacion	ScoringService.java
Context/Provider	Estructural	frontend	AuthContext.jsx
Circuit Breaker	Arquitectonico	bff-service	CircuitBreakerConfig.java

3. Patron Factory Method

3.1 Problema

El microservicio ms-mascotas necesita crear dos tipos de reporte (PERDIDO / ENCONTRADO). Sin una factory, cada componente que crea reportes deberia conocer que clase concreta instanciar, acoplando la logica de negocio a los detalles de implementacion.

3.2 Solucion - ReporteFactory

La clase ReporteFactory centraliza la creacion. El cliente solo indica el tipo en el DTO y la factory decide que subclase instanciar:

```
// ms-mascotas/.../factory/ReporteFactory.java
public Reporte crear(ReporteRequestDTO dto) {
    return switch (dto.getTipo()) {
        case PERDIDO    -> new ReportePerdido(dto);
        case ENCONTRADO -> new ReporteEncontrado(dto);
    };
}
```

3.3 Jerarquia de clases

Clase	Tipo	Descripcion
Reporte	Abstracta	Clase base con campos comunes (id, fecha, descripcion, foto)
ReportePerdido	Concreta	Anade fecha de perdida y ultima ubicacion conocida
ReporteEncontrado	Concreta	Anade fecha de hallazgo y ubicacion donde fue encontrada
ReporteFactory	Factory	Decide que subclase instanciar segun dto.getTipo()

3.4 Ventajas

- Agregar un nuevo tipo de reporte solo requiere un nuevo case en la factory.
- ReporteService no depende de ninguna clase concreta (abierto/cerrado - OCP).
- Facilita el testing: se puede inyectar mocks de la factory.

4. Patron Observer (Event-Driven)

4.1 Problema

Al crearse un reporte en ms-mascotas, otros microservicios deben reaccionar. Si ms-mascotas llamara directamente a esos servicios crearia acoplamiento estrecho y puntos de fallo unicos que degradan la resiliencia del sistema.

4.2 Solucion - RabbitMQ como canal de eventos

ms-mascotas publica un ReporteNuevoEvent en RabbitMQ. Los listeners de los demas servicios reaccionan de forma completamente desacoplada:

```
// EventPublisher.java
rabbitTemplate.convertAndSend(
    "sanos.exchange", "reporte.nuevo", event
);

// GeolocalizacionListener.java
@RabbitListener(queues = "geo.queue")
public void onReporteNuevo(ReporteNuevoEvent event) { ... }
```

4.3 Flujo de eventos

Paso	Productor	Routing Key	Consumidor
1	ms-mascotas / EventPublisher	reporte.nuevo	ms-geolocalizacion
2	ms-mascotas / EventPublisher	reporte.nuevo	ms-coincidencias
3	ms-geolocalizacion / GeoService	geo.completado	ms-coincidencias

5. Patron Proxy (BFF-Service)

5.1 Problema

El frontend no debe comunicarse directamente con cada microservicio. Eso expondría URLs internas, duplicaría la lógica de autenticación en el cliente y dificultaría agregar funcionalidades transversales (logging, throttling, circuit breaker).

5.2 Solución - bff-service como Proxy único

El BFF-Service actúa como proxy transparente: recibe todas las peticiones del frontend, valida el token JWT, y las redirige al microservicio correcto:

```
// AuthProxyService.java
public ResponseEntity login(Map credenciales) {
    return restTemplate.postForEntity(
        authServiceUrl + "/api/auth/login",
        credenciales, Map.class
    );
}
```

5.3 Rutas expuestas por el BFF (puerto 8080)

Prefijo BFF	Microservicio destino	Puerto interno
/api/auth/**	auth-service	8081
/api/reportes/**	ms-mascotas	8082
/api/geo/**	ms-geolocalizacion	8083
/api/coincidencias/**	ms-coincidencias	8084

6. Patron Strategy

6.1 ScoringService - ms-coincidencias

El algoritmo de puntuación de coincidencias encapsula cuatro criterios ponderados. Al implementarlo como Strategy, los pesos pueden cambiarse sin tocar el controlador ni el listener:

Criterio	Herramienta	Peso
Similitud del nombre	FuzzyWuzzy ratio()	40%
Similitud de descripción	FuzzyWuzzy partial_ratio()	30%
Distancia geográfica	Formula Haversine	20%
Diferencia temporal	ChronoUnit.DAYS	10%

```
// ScoringService.java
public double calcularPuntaje(CandidatoDTO c, ReporteNuevoEvent r) {
    double nombre = fuzzy.ratio(c.getNombre(), r.getNombre()) / 100.0;
    double desc = fuzzy.partialRatio(c.getDescripcion(), r.getDescripcion()) / 100.0;
    double dist = calcularDistancia(c.getLat(), c.getLon(), r.getLat(), r.getLon());
    double tiempo = calcularTiempo(c.getFecha(), r.getFecha());
}
```

```

    return 0.40*nombre + 0.30*desc + 0.20*dist + 0.10*tiempo;
}

```

6.2 ClusteringService - ms-geolocalizacion

Aplica el algoritmo K-means para agrupar reportes geograficamente. La encapsulacion en ClusteringService permite sustituir K-means por DBSCAN sin impactar al GeoController.

7. Patron Context/Provider (Frontend React)

El estado de autenticacion (token JWT, datos del usuario, funciones login/logout) necesita ser accesible desde multiples componentes sin prop-drilling.

```

// frontend/src/context/AuthContext.jsx
export const AuthContext = createContext();

export function AuthProvider({ children }) {
  const [user, setUser] = useState(null);
  const login = async (creds) => { /* llama al BFF */ };
  const logout = () => { setUser(null); };
  return (
    <AuthContext.Provider value={{ user, login, logout }}>
      {children}
    </AuthContext.Provider>
  );
}

```

8. Patron Circuit Breaker (Resilience4j)

Si un microservicio aguas abajo falla o se vuelve lento, el BFF-Service puede colapsar acumulando threads bloqueados. El Circuit Breaker detecta el fallo y retorna un fallback inmediato, protegiendo la estabilidad:

Estado	Descripcion	Condicion de transicion
CLOSED	Trafico normal, todas las llamadas pasan	Default
OPEN	Bloquea llamadas, retorna fallback inmediato	50%+ fallos en ventana deslizante
HALF_OPEN	Permite llamadas de prueba para recuperacion	Tras timeout configurado

9. Arquetipos Maven

9.1 Spring Boot Starter Parent (arquetipo oficial)

Todos los microservicios heredan de spring-boot-starter-parent 3.2.5, el arquetipo oficial que provee gestion de versiones, plugins de compilacion, configuracion de JaCoCo y empaquetado como fat-jar.

9.2 Arquetipo personalizado - sanos-y-salvos-microservice-archetype

Ubicado en /maven-archetypes/sanos-y-salvos-microservice-archetype. Genera la estructura base de cualquier nuevo microservicio del ecosistema:

- Application.java con @SpringBootApplication
- JwtAuthFilter.java — validacion de tokens JWT en cada request
- SecurityConfig.java — cadena de filtros Spring Security (stateless)
- RabbitMQConfig.java — TopicExchange, Queue y Binding preconfigurados
- application.yml con placeholders para puerto, datasource y rabbitmq
- ApplicationTests.java con test contextLoads()
- pom.xml con todas las dependencias del ecosistema (web, jpa, amqp, security, jjwt, postgresql, jacoco)

9.3 Generacion de nuevo microservicio

```
# Instalar el arquetipo en el repositorio local Maven
cd maven-archetypes/sanos-y-salvos-microservice-archetype
mvn install

# Generar nuevo servicio
mvn archetype:generate \
-DarchetypeGroupId=com.sanosysalvos \
-DarchetypeArtifactId=sanos-y-salvos-microservice-archetype \
-DarchetypeVersion=1.0.0 \
-DgroupId=com.sanosysalvos \
-DartifactId=ms-nuevo \
-DserviceName=Nuevo \
-DservicePort=8085 \
-DdatabaseName=nuevo_db
```

10. Justificacion de patrones seleccionados

Patron	Alternativa descartada	Razon de eleccion
Factory Method	if/else en el servicio	Viola OCP; agregar tipos requiere modificar codigo existente
Observer (RabbitMQ)	Llamadas REST sincronas directas	Desacoplamiento total; resiliencia ante fallos parciales
Proxy (BFF)	Frontend llama directo a cada microservicio	Un solo punto de entrada facilita auth, logging y versionado de API
Strategy (Scoring)	Algoritmo directo en el listener	Permite cambiar pesos/criterios sin tocar el flujo de eventos
Circuit Breaker	Sin resiliencia	Evita cascada de fallos; mejora UX con respuestas rapidas de fallback